SOFIA UNIVERSITY ST. KLIMENT OHRIDSKI

FACULTY OF ECONOMICS AND BUSINESS ADMINISTRATION

Master thesis

Dynamic Pricing Optimization using Bernoulli Multi-Armed Bandit Algorithms

Balancing Exploration and Exploitation for Regret Minimization

Author:

Slav Yolov

5EB3100235

Supervisor:

Assoc. Prof. Angel Marchev Jr.

Sofia
February 2024

Contents

1 Introduction 3
2 Theoretical aspects of modeling prices using Reinforcement Learning 6
2.1 Reinforcement learning 7
2.1.1 Notation 7
2.1.2 Action 7
2.1.3 Reward 8
2.1.4 State 8
2.1.5 Policy 8
2.1.6 Value function 9
2.1.7 Number of actions 10
2.1.8 Agent-environment interaction 10
2.1.9 Multi armed bandits 11
2.1.10 Exploration- exploitation tradeoff 13
2.1.11 Stationary and non-stationary bandits 14
2.1.12 Variations of the multi armed bandit 14
2.2 Price optimization fundamentals 16
2.2.1 Elasticity measures 16
2.2.2 Price response functions 18
2.3 Optimization 23
3 Methodology 24
3.1 Introduction 24
3.2 Background 24
3.3 Data 24
3.3.1 Variables 25
3.3.2 Data Anonymization 25
3.3.3 Time Frame 26
3.3.4 Quality Assurance 26
3.3.5 Ethical Considerations 26
3.4 Algorithms 26
3.4.1 Greedy algorithm 27
3.4.2 Epsilon-Greedy algorithm 28
3.4.3 Thompson Sampling 29
3.4.4 Upper Confidence Bound (UCB1) 31
3.5 Demand model 33
3.5.1 Demand function 34

3.5.2 Reward function 37
3.5.3 Optimal price 38
3.6 Metrics for evaluation 39
3.6.1 Regret 39
3.6.2 Arms allocation 40
3.6.3 Reactivity 40
3.7 Price candidates 40
4 Empirical research 42
4.1 Simulation loop 42
4.2 First phase: Algorithms performance in stationary environment 43
4.3 Second phase: Non-stationary environment 46
5 Conclusions 53
References 56

1 Introduction

Dynamic pricing, a strategy characterized by real-time price adjustments based on market conditions and consumer behavior, has gained prominence across various industries including e-commerce, transportation, and hospitality. This adaptive pricing approach allows businesses to optimize revenue by aligning prices with fluctuations in demand and competitive dynamics.

The optimization problem in dynamic pricing can be framed as a multi-armed bandit problem, a classic problem in reinforcement learning, where an agent must choose between multiple price points (arms), each with an unknown reward probability following a Bernoulli distribution. The goal is to minimize the total regret obtained over a series of actions by efficiently balancing exploration (trying different arms) and exploitation (leveraging knowledge gained to choose the best arm).

The main objective of this thesis is to utilize Bernoulli multi-armed bandit algorithms to optimize price selection in the context of one of the biggest online food retailers in Europe. To achieve this goal, several key tasks must be undertaken:

1. The implementation of the four selected Bernoulli multi-armed bandit algorithms—greedy, epsilon-greedy, Thompson sampling, and UCB1—is paramount. These algorithms are adapted to suit the context of dynamic pricing optimization within the online food retailing domain.
2. Developing a resilient global demand function and reward mechanism is crucial to steer the algorithms toward identifying the most optimal arm (price).

3. The design of appropriate experiments or simulations is crucial for evaluating the performance of the implemented algorithms. This entails defining the parameters and conditions under which the algorithms will operate, including factors such as the range of price points, purchase probabilities, and the duration of the pricing experiments.
4. Once the simulations are completed, each algorithm undergoes rigorous evaluation based on predefined metrics such as regret, reactivity, and arm allocation.
5. The final task involves interpreting the findings and drawing conclusions based on the empirical results.

While the emphasis is placed on the implementation and evaluation of Bernoulli multi-armed bandit algorithms, certain methods are deemed out of scope for the current thesis. Traditional pricing optimization techniques, such as linear pricing models or heuristic-based approaches, fall outside the scope of this research. Additionally, more complex multi-armed bandit variants, such as contextual bandits or non-stationary bandits, are not considered within the scope of this study. Instead, the thesis aims to examine the performance and comparative effectiveness of the selected Bernoulli multi-armed bandit algorithms in the specific context of online food retailing, where data is sourced from the retailer database, and a single stock-keeping unit is analyzed.

The empirical research designs an environment encompassing stationary and non-stationary environments to gauge algorithm performance under different conditions. Stationary settings, characterized by stable parameters over time, provide insights into algorithms' behavior in static environments. In contrast, non-stationary settings, characterized by dynamic purchase probabilities

and changing competitive landscapes, test the algorithm's adaptability to evolving conditions. Through comprehensive experimentation, the empirical research aims to identify the most suitable Bernoulli multi-armed bandit algorithms for optimizing dynamic pricing in online food retailing. The comparison across various metrics, including regret, reactivity, and arm allocation, facilitated a nuanced understanding of each algorithm's strengths and limitations.

2 Theoretical aspects of modeling prices using Reinforcement Learning

To maximize the total expected profit in a competitive market, the seller must balance two key objectives: the gross margin for the given price and the conversion rate, defined as the probability that a customer will purchase the product at that price. Since knowledge about the conversion rate can only be acquired by offering the product at different prices, this inherently involves managing the trade-off between exploration and exploitation. In other words, there is a need to balance optimizing decisions based on current knowledge of the conversion rate and acquiring new knowledge about it. One effective approach to tackle this challenge is by employing the multi-armed bandit problem formulation.

In concrete terms, a multi-armed bandit is an online learning process characterized by a set of successive rounds where an agent, also referred to as a learner, interacts with the environment. During each round or time step, the agent selects an action from the available options, and in response, the environment provides a reward. The primary objective is to maximize the cumulative reward obtained over time by identifying the most favorable arm.

Another characteristic of online learning is the absence of distinct training and testing phases, as seen in supervised learning. In online learning algorithms, these two phases merge into one continuous process. During each round, the algorithms receive an instance, make a prediction based on it, and subsequently receive the true label, obtaining a loss as feedback for this prediction. This iterative process allows for real-time adaptation and continual improvement of the model without the need for predefined training and testing splits.

Now, delving into the theoretical underpinnings of our study, we shift our focus to explore the foundational concepts that inform the application of Bernoulli multi-armed bandit algorithms in the context of online food retailing.

2.1 Reinforcement learning

Reinforcement learning involves discovering optimal actions in various situations to maximize a numerical reward signal, without explicit guidance on which actions to take. Instead of being instructed, the learner explores actions to identify those that result in the highest rewards through trial-and-error process. In complex scenarios, actions may influence not only immediate rewards but also subsequent situations and, consequently, all future rewards. The key characteristics that set reinforcement learning apart from supervised and un-supervised learning are the iterative process of trial-and-error search and the consideration of delayed rewards (Sutton & Barto, 2018, p.3).

2.1.1 Notation

The notation aligns with that used in Sutton & Barto (2018)

2.1.2 Action

The process of deciding between available options and choosing one is known as action. In the case of the multi-armed bandit (MAB) problem, it entails choosing and playing one of the arms of slot machines. The action is denoted as A_t where t is the time step.

2.1.3 Reward

Whenever an action is executed, a corresponding reward is emitted. The reward associated with a specific action is a stochastic value, drawn from an underlying probability distribution unique to that action. Consequently, each instance of taking an action result in a potentially different reward value. If a particular action is repeated multiple times, a more accurate estimate of the true mean of the reward can be computed. The reward serves as a guiding factor in the learning process, aiming to identify optimal actions to maximize the cumulative reward. The reward is denoted as $\mathrm{R_t}$ (Sutton & Barto, 2018, p. 53)

2.1.4 State

This is the information available to the agent regarding the underlying environment. At this stage, it is important to mention that the MAB problems are stateless (Barrachina-Muñoz et al., 2021). This means that the distribution of the reward is fixed for each arm. The reward is independent of which arm was previously pulled.

2.1.5 Policy

According to Sutton & Barto (2018), the policy represents the behavior of an agent in its environment. It serves as a mapping from perceived states of the environment to actions to be taken when encountering those states. In fact, this is a mapping from actions to states. There are two primary types of policies, deterministic and stochastic.

- A deterministic policy, denoted as $\pi : S \rightarrow A$, maps each state $s \in S$ to a specific action $a \in A$ that the agent will choose while in that state.
- On the other hand, a stochastic policy, denoted as $\pi : S \, x \, A \rightarrow [0, 1]$, provides a probability distribution over actions for each state-action pair. For every state $s \in S$ and action $a \in A$, it yields the probability $\pi(a \mid s)$ that the agent selects action a while in state s.

2.1.6 Value function

Most of the reinforcement learning algorithms rely on estimating value functions, which are functions of states (or state-action pairs). These functions estimate the desirability for an agent to be in a particular state (or to take a specific action in a given state). The concept of "desirability" is determined by considering the future rewards the agent can anticipate, or more precisely, by evaluating the expected return. Naturally, the rewards an agent can anticipate in the future are contingent upon the actions it chooses to take (Sutton & Barto, 2018, pp. 62-64). In Figure 1, the state-value and the action-value functions are described.

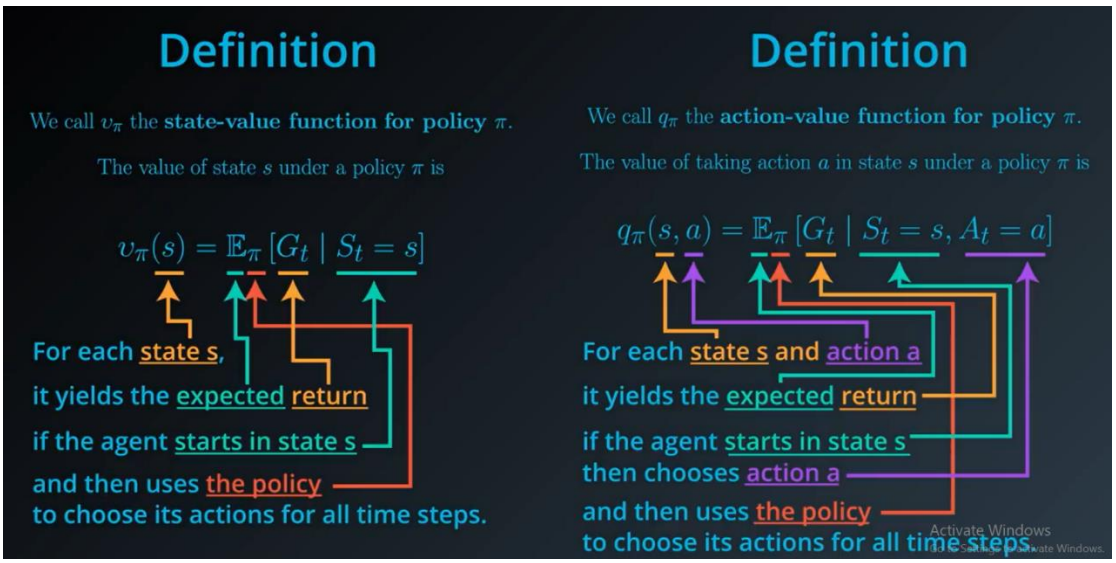

Figure 1. Value functions, Reinforcement Learning. (n.d.).

2.1.7 Number of actions

The set of actions is denoted as ‘k’. In the MAB problem, this is the total number of arms to choose from.

2.1.8 Agent-environment interaction

The algorithm responsible for learning and making decisions is called an agent. The environment encompasses everything outside the agent with which it interacts. This interaction is ongoing, involving the agent's selection of actions and the environment's response to these actions, introducing new situations to the agent and reward (Sutton & Barto, 2018, pp. 47-48). A common interface of the agent-environment interaction is available in Figure 2.

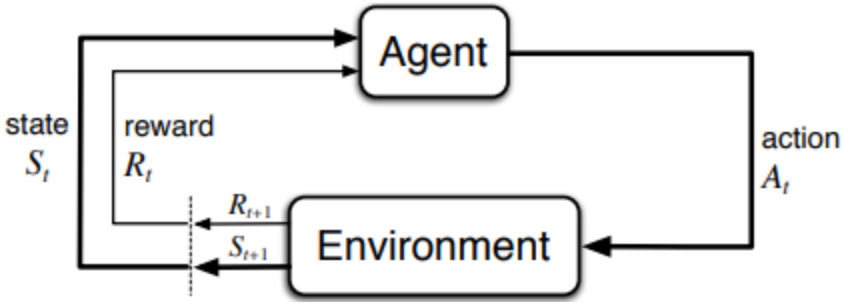

Figure 2. Interaction between agent and environment in a Markov decision process, Sutton and Barto (2018).

To elaborate, the agent and environment engage in interactions at discrete time steps, denoted as t = 0, 1, 2, 3, and so forth. At each time step t, the agent is provided with a representation of the current state of the environment ($S_t \in S$). Based on this information, the agent selects an action A_t from the set of actions. One time step later, as a result of the chosen action, the agent receives a numerical reward, R_{t+1}, within the range R. Additionally, the agent transitions to a new state, S_{t+1} . The combination of the Markov Decision Process (MDP) and the agent's actions results in a sequential trajectory that unfolds in the following manner:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3 \ldots S_n, A_n, R_n \quad (1)$$

2.1.9 Multi armed bandits

The concept of multi-armed bandits (MAB) represents a widely recognized challenge encountered when an agent is tasked with making successive decisions from a predetermined set of options. The underlying assumption is that within this set of options, there is an optimal choice that can only be determined through iterative experimentation.

Robbins (1952) introduced the MAB. The concept originated from a theoretical casino scenario in which a gambler engages in playing a group of slot machines commonly known as one-armed bandits. The underlying assumption is that certain machines are more likely to yield wins compared to others.

The agent picks an action from K different actions. After each an action A the agent receives a reward R. Rewards are determined by the chosen action and are selected from a stationary or non-stationary probability distribution. The goal of the agent is to maximize the total expected reward (Kuleshov & Precup, 2014). The expected reward of an action is denoted as $q(a)$ where a is the action at each time step A_t . As seen in (2) taken from Sutton & Barto (2018, p.26).

$$q(a) = \mathrm{E}\,[R_t \mid A_t = a] \tag{2}$$

The expected reward associated with a particular action is referred to as the value of that action, denoted as $q(a)$, where a represents the action chosen at time step t . This statement implies that by consistently repeating action a and averaging the obtained reward values results in acquired understanding of the true value $q(a)$ - the mean reward for action a.

A method for estimating the expected total reward involves calculating the average of the received rewards (3) (Sutton & Barto (2018, p.27):

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}} \quad (3)$$

Although the classic illustration of the Bandit problem revolves around a casino scenario, its essence lies in encapsulating a genuine dilemma through an intuitive hypothetical situation. This hypothetical scenario has inspired numerous variations, each tailored to mirror real-world predicaments sharing similar characteristics yet featuring unique nuances that necessitate slightly different problem-solving approaches.

2.1.10 Exploration- exploitation tradeoff

The primary challenge in the MAB problems revolves around the inherent tradeoff between exploration and exploitation. Exploration entails seeking information about the potential possibilities within the available choices, while exploitation involves maximizing rewards based on the assumption of the current best choice. However, navigating this tradeoff is inherently complex, as optimal approaches are contingent upon the specific scenario's specifications. This complexity often results in a spectrum of potential adjustments to further optimize policies tailored to the given context. The intricacies of the MAB scenario, including its overall complexity, the number of bandits in play, the probability of reward per pull, reward variance, and the temporal nature of the problem, all significantly influence the optimal strategies. Furthermore, circumstantial properties associated with the given MAB problem introduce additional layers of complexity (Yang et al., 2022).

2.1.11 Stationary and non-stationary bandits

In the context of multi-armed bandits, the term "stationary probability distribution" refers to the assumption that the underlying probabilities associated with each arm remain constant over time. This assumption simplifies the problem and allows for the application of certain algorithms. The concept of a stationary probability distribution contrasts with a non-stationary scenario, where the probabilities of different arms can change over time (Taywade et. Al, 2023).

The assumption of stationarity is often made for the sake of mathematical tractability and ease of analysis. Many traditional bandit algorithms, such as the epsilon-greedy algorithm or the Upper Confidence Bound (UCB) algorithm, are designed under the assumption of a stationary environment.

2.1.12 Variations of the multi armed bandit

A diverse array of the multi-armed bandit problem has been formulated, each introducing unique complexities or considerations to address specific scenarios and challenges in reinforcement learning. These variants cater to a wide range of applications, reflecting the adaptability of the basic framework to diverse real-world situations. Without delving into extensive details, the current section lists some of the well-known variants.

2.1.12.1 Adversarial bandit

The adversarial bandit problem is a variant of the classical multi-armed bandit problem where the rewards for choosing different arms

are chosen by an adversary rather than being fixed or drawn from a stationary probability distribution. In the adversarial bandit setting, the rewards associated with each arm can change arbitrarily at each time step, making it a more challenging and dynamic problem (Auer et al., 2000).

2.1.12.2 Contextual bandit

A Contextual Bandit is an extension of the classic multi-armed bandit problem that introduces the idea of context or additional information. In a contextual bandit setting, each arm (action) is associated with a context, and the goal is to learn a policy that can map contexts to the best possible action to maximize cumulative reward. Contextual bandit algorithms leverage techniques from machine learning, such as supervised learning, to model the relationship between contexts and expected rewards. The challenge in contextual bandit problems is to balance the exploration-exploitation tradeoff. The algorithm needs to explore different actions to learn about their rewards in different contexts while also exploiting its current knowledge to choose actions that are likely to yield high rewards (Slivkins, 2019, p.93).

2.1.12.3 Restless bandit

According to Liu et al. (2023), restless bandits are commonly characterized as bandits in which the "reward distribution" changes over time. In the seminal paper by Whittle (1988), non-stationary bandits are specifically termed “restless bandits”, denoting bandits where "the objects continue to change state even when they are not

being operated." Unlike the stationary bandits, where the reward probabilities remain constant, non-stationary bandits introduce dynamics, making the optimal arm potentially variable.

2.1.12.4 Bernoulli (Binary) bandit

A Bernoulli bandit refers to a specific type of multi-armed bandit problem where each arm's reward distribution follows a Bernoulli distribution. In this context, each action results in a binary outcome typically representing success or failure, with probabilities associated with each outcome. The challenge is to learn and exploit the arms that yield higher success probabilities over time (Lattimore and Szepesvári, 2020). The Bernoulli bandit reward is given based on the arm probability. The reward of 1 is given with probability Θ_k while the 0 reward is given with probability of 1 - Θ_k (Russo et al., 2018, p.13)

2.2 Price optimization fundamentals

2.2.1 Elasticity measures

There are multiple simple ways to compute the price elasticity of demand (PED). The simple measures usually require two datapoints to calculate the elasticity between period t and $t + 1$.

One common way to calculate the PED (4) is by taking the ratio of the percentage change in quantity to the percentage change in price (Phillips, 2005, p.43).

$$E_{\langle P \rangle} = \frac{\Delta Q}{\Delta P} \tag{4}$$

Where:

- ΔQ - denotes the percentage change in quantity (q2 – q1).
- ΔP – denotes the percentage change in price (p2 – p1).

It is important to note that the value of $E_{(p)}$ is not invariant to the ordering of the data points. As seen in (5), this limitation is fixed by applying the arc elasticity $E_{(d)}$ (Phillips, 2005, p.44)

$$E_{(d)} = \frac{p2 + p1}{q2 + q1} * \frac{\Delta Q}{\Delta P} \tag{5}$$

In the economic books Mankiw (2016), Krugman (2012), and other the interpretation of elasticity coefficients is as follows:

- When the price elasticity of demand (PED) exceeds 0, it denotes elastic demand, implying that the quantity demanded responds to changes in price.
- If PED is below 1, it signifies inelastic demand, indicating that the quantity demanded shows little responsiveness to price changes.
- A PED of 1 denotes unit elastic demand, where the percentage change in quantity demanded matches the percentage change in price.

2.2.2 Price response functions

A crucial component of any price and revenue optimization analysis is the price-response function, Figure 3. The function is usually denoted as d(p) which specifies how the product demand (d) changes as a function of the price (p) (Phillips, 2005, p. 38).

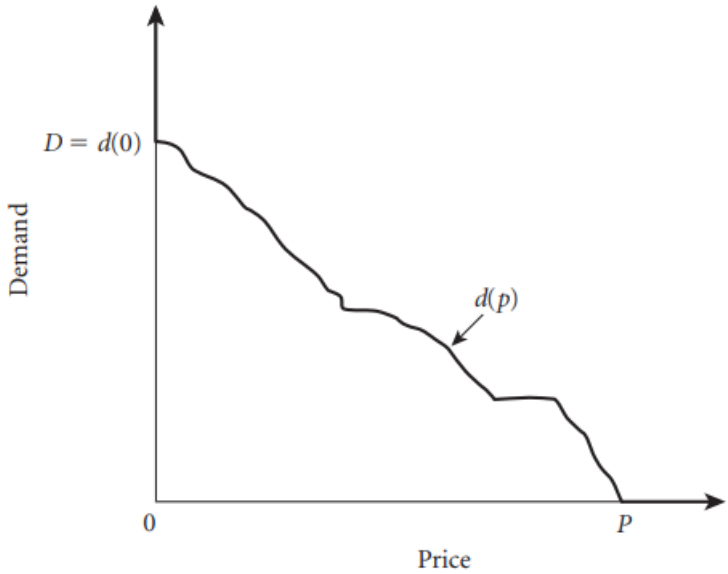

Figure 3. Price-response function, Philips (2005).

According to Phillips (2005, p.40), the price-response functions are positive, continuous, nonnegative, differentiable, and downward-sloping. Supporting these, in many cases and under typical circumstances, an increase in price leads to a decrease in demand, while a price decrease tends to increase demand.

Selecting and estimating the appropriate price-response function is the most challenging aspect of any pricing analytics project (Bodea & Ferguson, 2014, p.143). Bodea and Ferguson (2014, pp. 143–146) outline four common price response functions, which are further elaborated in the subsequent section.

2.2.2.1 Linear function

If the demand curve is assumed to be linear, the well-known linear regression equation can be used to find the price response function (6).

$$d(p) = D - mp \tag{6}$$

Where D is the intercept term, p is the independent variable (price), and m = $\frac{D}{P}$ is the slope (elasticity). The relationship between the price and demand is inverse – as the price increases the demand decreases (Phillips, 2005, p.49).

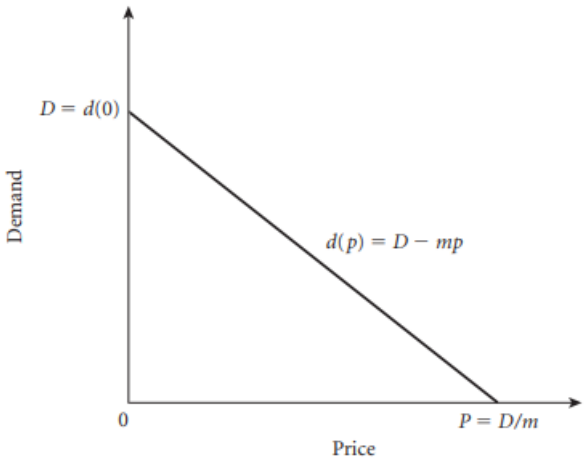

Figure 4. Linear price response curve, Phillips (2005)

As seen in Figure 4, the linear price-response function displays a uniform relationship (the change in demand resulting from a 20-cent price increase remains consistent, regardless of the initial base price). This assumption is unrealistic, particularly in markets with competitors that offer comparable products. (Phillips, 2005, p.49)

2.2.2.2 Constant elasticity function

Constant elasticity assumes uniform elasticity across the different price points. In (7) shows the constant elasticity price response function.

$$d(p) = Cp^{-\epsilon} \tag{7}$$

Where ϵ > 0, C is a constant that is equal to the demand at the lowest price. For example, if a product is selling 5000 units at the lowest price 1 € then d(1) = 5000, from where C = 5000. The function has a downward slope, but it is never finite and therefore the demand cannot drop to zero. Due to these factors, constant elasticity is not a proper global approximation for price response. (Phillips, 2005, p.49). The function is usually estimated using linear regression models that are transformed with logarithms on both sides (log-log models) (Carilli, 2020).

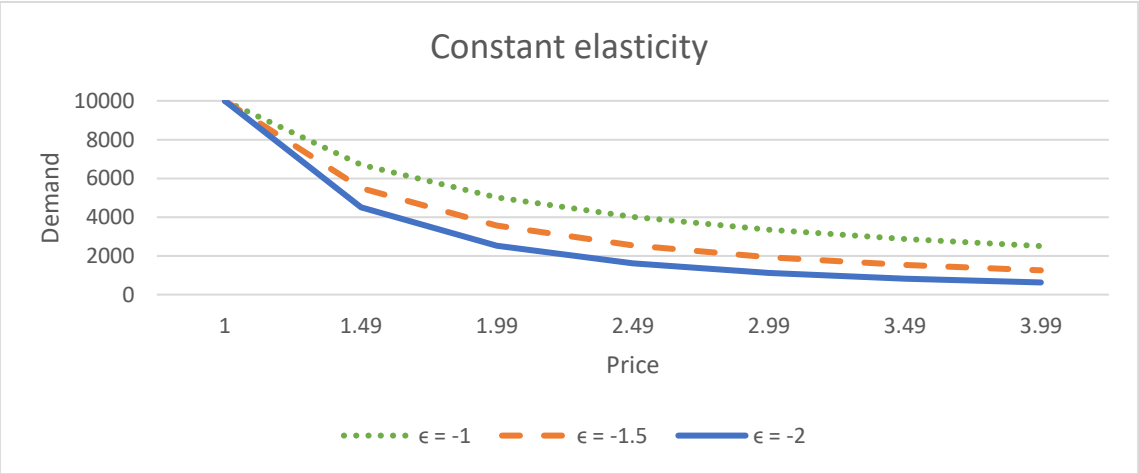

Figure 5. Constant elasticity at different levels of ϵ, C = 10000, Source: Author (2024)

2.2.2.3 Power function

As shown in (Phillips, 2005, p.50) The power function is a demand function with a reversed S-shape (8).

$$d(p) = Cp^{-\epsilon} \tag{8}$$

Where C > 0, α and β are estimated by fitting the equation to the. The function is displayed in Figure 6 and as evident from it, higher values of β show markets that are highly responsive to changes in price, and vice versa. The α parameter moves the curves along the horizontal axis. (Bodea & Ferguson, 2014, p.145)

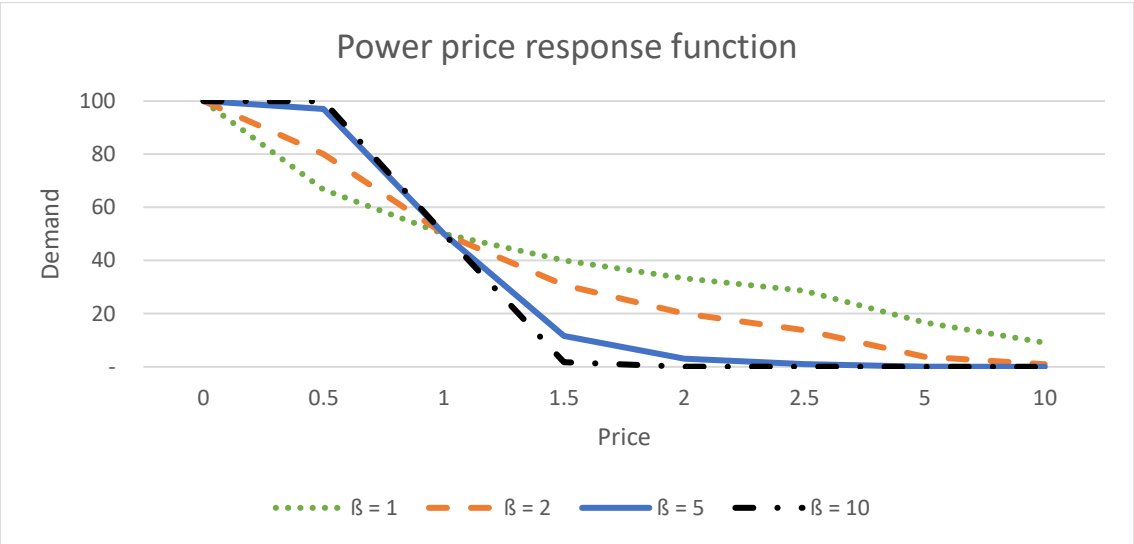

Figure 6. Power function at different levels of β, C = 100, α = 1, Source: Author (2024)

2.2.2.4 Logit function

The logit price-response function (Figure 7) exhibits a sigmoidal, reversed S-shaped curve (2005, p.53). When setting prices at an exceptionally low level, we observe a considerable increase in demand; however, the rate of change in demand is gradual with price adjustments. Within the region of the "market price," demand exhibits high sensitivity to our pricing—minor fluctuations in price can result in significant shifts in demand. Conversely, at elevated price points, demand remains low and experiences slow changes even with further increases in prices.

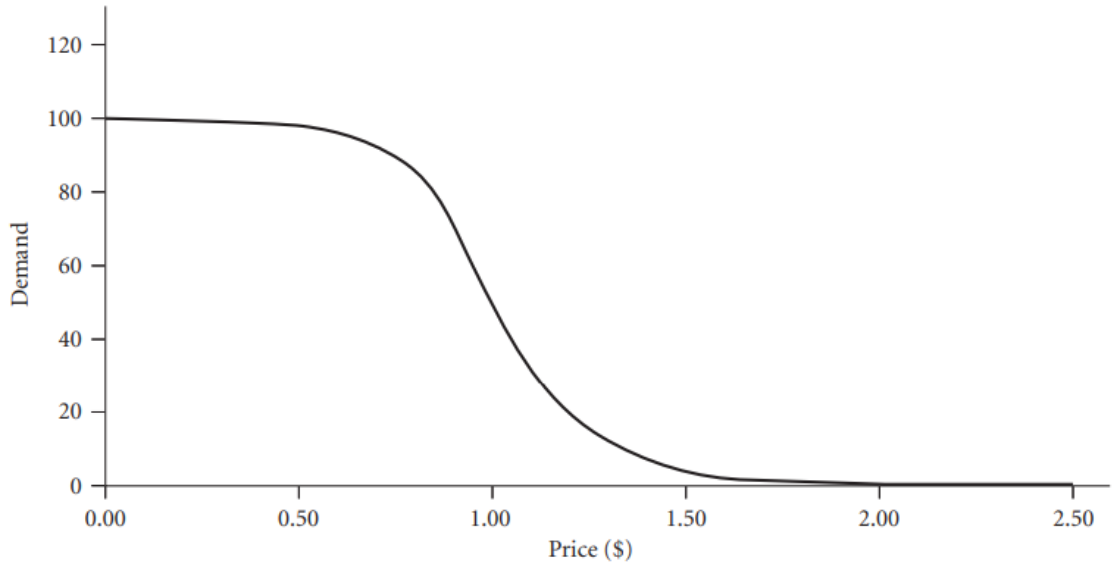

Figure 7. Reversed sigmoid (S-shape) price-response function, Phillips (2005)

In (9), a common price-response curve of this form is given by Phillips (2005, p.53)

$$d(p) = \frac{Ce^{-(a+bp)}}{1 + e^{-(a+bp)}} \tag{9}$$

Having the following values for our parameters (C, $\hat{p}$, β) : C = 115, $\hat{p}$ is fixed at 80, $\beta\,(0.5, 0.15, 0.01)$. The α parameter can be obtained by calculating the equation: $\hat{p} = -\alpha/\beta$. Using these parameters and set of prices the following demand curves can be constructed (Figure 8).

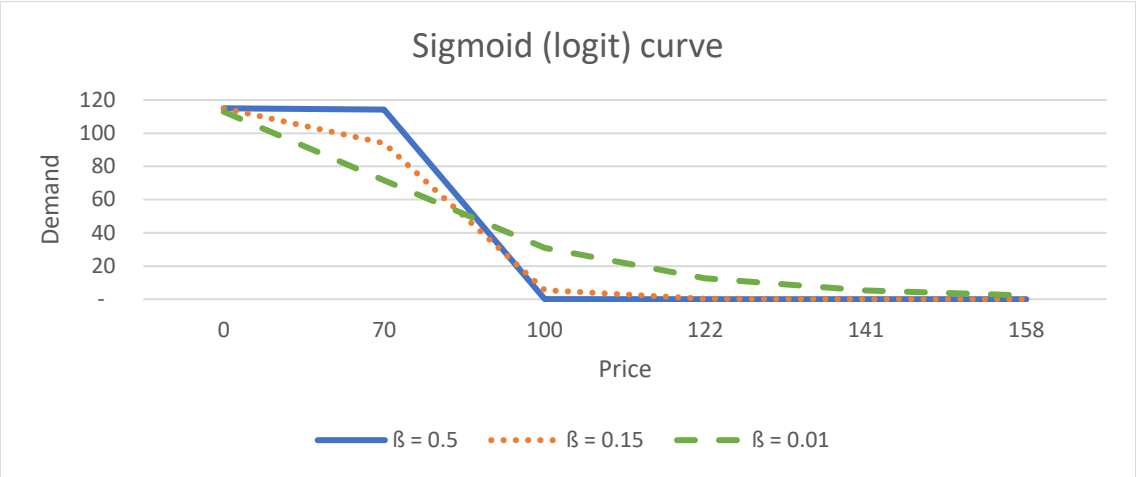

Figure 8. Logit demand curve, Source: Author (2024)

2.3 Optimization

The fundamental objective of the pricing optimization problem is to maximize the total profit that represents the total generated revenue minus the total cost. While pricing and revenue optimization extensively utilize constrained algorithms, it is crucial to emphasize the challenge, if not the impossibility, of pinpointing the truly "optimal" price. This is because most if not all algorithms are based on solving simplified models of the underlying problem. (Phillips, 2005, p.34)

Henceforth, within this master's thesis, the term "optimal" price is employed to denote the optimal price in consideration of the chosen algorithm and underlying assumptions.

3 Methodology

3.1 Introduction

In the context of pricing optimization, formulating the problem as a multi-armed bandit (MAB) problem involves treating different pricing strategies or price points as "arms" and seeking an optimal strategy to maximize revenue or other key performance indicators. The problem is of a discrete kind as the arms can only take distinct values.

3.2 Background

Traditional pricing strategies often involve fixed pricing models, which may not fully exploit the potential for revenue maximization in a constantly evolving market. The multi-armed bandit problem, derived from the field of probability theory and decision-making, provides an effective model for dynamically allocating resources among competing options with uncertain outcomes.

3.3 Data

The data collection process for this study focuses on gathering pricing-related information for a single Stock Keeping Unit (SKU) from an online retailer. This section outlines the sources, methods, and anonymization techniques employed to acquire weekly sales, views, and conversion rate data for the year 2023. The data source is the internal database of an online retailer, containing detailed information on product performance metrics. The dataset encompasses a time

frame that allows for a comprehensive analysis of weekly fluctuations in sales, product views, and conversion rates.

3.3.1 Variables

The variables in the input data are:

- **Sales:** Weekly sales that represent the total number of units for the specified SKU sold during a given week.
- **Views:** Weekly product views denote the total number of times SKU's product page was accessed by online visitors.
- **Conversion Rate:** The conversion rate is calculated as the ratio of weekly sales to product views, providing insights into the percentage of views that resulted in a purchase.
- **Stock keeping unit (sku_id)** – unique product identification.
- **Week** – calendar week
- **Year** – calendar year

3.3.2 Data Anonymization

To safeguard confidentiality and ensure the privacy of both individuals and the retailer, a rigorous anonymization process was implemented on the collected data. This process involved the removal or replacement of personally identifiable information, such as customer details, with anonymized identifiers. Consequently, the analysis focuses exclusively on aggregate product-level metrics.

3.3.3 Time Frame

The data spans a sufficient time frame (52 weeks), capturing weekly observations over a specified period. This timeline enables the analysis of trends, seasonality, and variations in pricing metrics over different weeks.

3.3.4 Quality Assurance

To ensure data accuracy and reliability, the data was validated through a quality assurance process. This included validating the consistency of data entries, checking for outliers, and confirming the integrity of the collected SKU-specific metrics.

3.3.5 Ethical Considerations

The data collection adheres to ethical standards, ensuring compliance with privacy regulations and the retailer's data usage policies. No personally identifiable information is included in the dataset, and the analysis is focused on aggregated product-level metrics.

3.4 Algorithms

In the pursuit of optimizing pricing strategies for a dynamic and uncertain market environment, the selection of appropriate algorithms plays a pivotal role. This section explores the rationale behind choosing three distinct algorithms: Greedy, Epsilon-Greedy, and Thompson Sampling. Each algorithm embodies unique

characteristics that align with the research objectives of maximizing revenue in fluctuating market conditions.

All models are constructed within the framework of the Bernoulli Bandit context.

3.4.1 Greedy algorithm

A Greedy algorithm, by definition, is a decision-making approach that consistently selects the best immediate, or local, solution at each step in the process (Figure 9). While its primary objective is to find the overall optimal solution for certain optimization problems, it should be noted that Greedy algorithms may occasionally yield less-than-optimal solutions in specific instances of other problems (*Greedy Algorithm*, n.d.). The Greedy approach is typically used as a baseline as this is the least-performing algorithm.

```
if no episodes played
    pick arm at random
else:
    choose the arm with the highest average reward
```

Figure 9. Pseudo-code greedy algorithm, Source: Author (2024)

As shown in Figure 10, the Greedy algorithm operates by consistently selecting the vertex with the largest number at each step. Starting from vertex 6, the algorithm makes a series of decisions based on comparing the numbers associated with neighboring vertices.

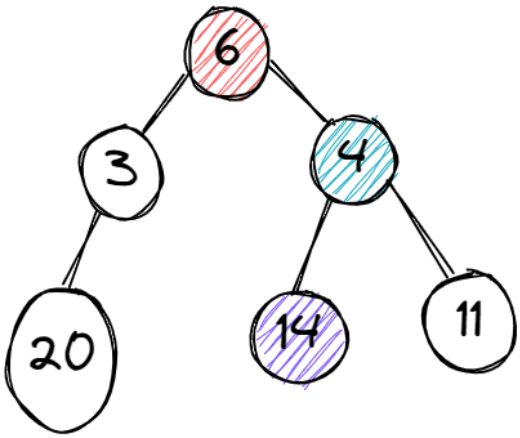

Figure 10. Example of decisions taken by the greedy algorithm (Alabi, 2023)

Decision 1: 6 → 4 or 3

- Greedy chooses 4 over 3 as it has a larger number.

Decision 2: 4 → 14 or 11

- Greedy selects 14 over 11, again favoring the larger number.

The algorithm terminates after this series of decisions, reaching vertex 14. However, it is crucial to highlight a vertex labeled 20, connected to vertex 3, which Greedy does not consider in its sequence of choices. The algorithm's criterion for immediate decisions relies solely on the immediate comparison of numbers associated with neighboring vertices. Due to this, the algorithm occasionally produces suboptimal solutions.

3.4.2 Epsilon-Greedy algorithm

As shown in Figure 11, the epsilon-greedy algorithm is a modification designed to address the primary limitation of the greedy approach. It incorporates a probability ε (epsilon), usually set to a small value, to randomly select an arm, fostering exploration.

Meanwhile, with a probability of 1−ε, it opts for the arm with the highest estimated reward, emphasizing exploitation. The strategy strives to attain improved long-term returns by finding a balance between random exploration and the exploitation of known rewards, offering a more nuanced approach compared to the greedy strategy.

```
p = random()

if p < ε:
    pull random arm
else:
    pull current-best arm
```

Figure 11. Pseudo-code Epsilon-greedy algorithm, Source: Author (2024)

3.4.3 Thompson Sampling

Thompson Sampling proposed in (1933) and made popular recently by Chapelle and Li (2011) is a Bayesian methodology that tackles the exploration-exploitation challenge by choosing arms probabilistically according to their posterior reward distributions. In situations where the rewards follow a Bernoulli distribution, which models binary outcomes such as success or failure, Thompson Sampling (TS) utilizes the Beta distribution as a conjugate prior. The pseudo-code of the algorithm is available in Figure 12.

```
Require: α, β prior parameters of a Beta distribution
  S_i = 0, F_i = 0,  ∀i. {Success and failure counters}
  for t = 1, ..., T do
    for i = 1, ..., K do
      Draw θ_i according to Beta(S_i + α, F_i + β).
    end for
    Draw arm î = arg max_i θ_i and observe reward r
    if r = 1 then
      S_î = S_î + 1
    else
      F_î = F_î + 1
    end if
  end for
```

Figure 12. Pseudo-code for Thompson sampling for the Bernoulli bandit problem, Chapelle and Li (2011)

The initial estimation of the likelihood that the arm will generate an output is known as prior probability. It denotes the probability of a particular event taking place before any evidence is gathered. In this context, the prior probability is characterized by the Beta distribution Beta (1,1). Subsequently, the algorithm updates the distribution parameters in response to the observed rewards: the alpha parameter is increased on success, while a failure augments the beta. In each iteration, Thompson Sampling randomly samples from the current Beta distribution associated with each arm and selects the one with the highest sampled value. This approach enables Thompson Sampling to dynamically adapt by considering collected rewards, effectively striking a balance between exploring uncertain arms and exploiting those recognized as rewarding (Russo et al., 2018).

In the thesis, the actual reward of interest is derived from the product of the basic reward, where success is assigned a value of 1 and failure a value of 0, and the current price under evaluation. Consequently, the implementation of Thompson Sampling undergoes a slight modification to account for this relationship. This adjustment

ensures that decisions are informed by the expected average revenue, thereby shifting the focus toward maximizing purchase probability.

3.4.4 Upper Confidence Bound (UCB1)

The Upper Confidence Bound 1 is a popular algorithm used to solve Bernoulli bandit problems. It starts by pulling each arm once to initialize the empirical mean rewards and counts. Next, at each time step the UCB1 selects an arm based on its empirical mean reward and the number of times it has been pulled. The UCB1 is usually calculated using a formula that balances the exploration of arms that have uncertain reward estimates with the exploitation of arms that have potentially high rewards. The arm with the highest reward at the current time step is selected. Accordingly, the empirical mean reward and the arm counter are updated. The process is repeated until a fixed number of time steps (Auer et.al, 2002). Equation (10) outlines the process for selecting the optimal action at a given time step.

$$A_t = argmax\left[\mu + C\,.\sqrt{\frac{2log(t)}{n_{i,t}}}\right] \tag{10}$$

Where,

- μ is the average reward for each arm at step t.
- C is a constant that balances the exploration-exploitation trade off. As the value of C increases, so does the level of exploration, while a decrease in C corresponds to a reduction in exploration.

- t accounts for the total rounds played up to step t
- n number of times each of the arms was selected.

The UCB1 algorithm (Figure 13) aims to balance the exploration of arms with uncertain rewards and the exploitation of arms with potentially high rewards. It does this by favoring arms that have not been explored much (i.e., have high uncertainty) and arms that have shown high rewards in the past. While UCB1 is a simple and effective algorithm, it assumes that the rewards are stochastic and stationary, meaning that the reward distribution of each arm does not change over time. If the reward distributions change significantly over time, UCB1 may struggle to adapt quickly, as it does not explicitly model changes in the environment (Kocsis and Szepesvári, 2006).

```
Play each arm once to get the initial reward values R_a
For each round:
        Play the action that maximizes (10)
        Observe the reward and update the R_a and N_a for the selected arm
```

Figure 13. Pseudo code UCB1 algorithm, Source: Author (2024)

3.4.4.1Normalization of the UCB1 algorithm

Furthermore, beyond the fundamental implementation of the algorithm, a normalization is incorporated. This step precedes the generation of the Upper Confidence Bound (UCB) values. The normalization process entails adjusting the rewards of each arm to a range between 0 and 1 by dividing each reward by the maximum reward. The aim of this normalization is to maintain a consistent

search range for the optimal hyperparameter C. Without normalization, adjustments to the search interval would be required based on the specific range of values involved.

3.5 Demand model

In the field of economics, conventional demand curves describe the correlation between a product's price and the corresponding quantity consumers are inclined to purchase. These curves typically exhibit a downward slope, reflecting the widely observed phenomenon where a price increase tends to result in a decrease in demand, and conversely, a decrease in price leads to an increase in demand.

The master thesis conceptualizes the demand curve with a different approach, juxtaposing price against probability. The rationale behind this choice lies in the realm of dynamic pricing where it is more meaningful to consider the probability of a sale at a specific price rather than making predictions about exact quantities. In such environments, each pricing iteration is an assessment of the likelihood of purchase. This can be modeled using the Bernoulli random variable with a probability 'p' that depends on a certain price.

The goal of the constructed demand model is to maximize the revenue. Therefore, the objective is not to find the price that generates the highest number of sales but to identify the price, when multiplied by the corresponding purchase probability, which has the highest expected return.

3.5.1 Demand function

Since the assumption that the connection between price and demand follows a linear pattern is not globally applicable. The master thesis adopts the approach to represent the price-demand relationship through logit (logistic) functions, which can capture this nuanced relationship better. The proposed demand function is given in (11)

$$y = \frac{\alpha}{1 + exp^{(ß \cdot p)}} \tag{11}$$

Where p is the price, α represents the maximum achievable probability of purchase, and ß moderates the demand sensitivity.

The value of α ranges logically between [1, 2] where at α = 1 the maximum probability is 50% at price 0, while at the value of 2, the maximum probability is 100%. The master thesis opts to use a fixed value of α = 1.90 which gives a maximum probability of 95% at price 0, assuming that there is a minor chance some customers to view the product but not buy it.

On the other hand, ß moderates the sensitivity of the demand to changes in price. When the ß parameter takes on a higher value, the demand curve becomes steeper, leading to a more rapid decline in purchase probabilities as the price increases. The challenge now lies in determining the optimal value for the parameter ß. In the thesis, this process involves solving the demand equation to find the value of ß.

To obtain the beta (β) parameter, the demand equation is solved using the following parameters:

- α = 1.9 (maximum probability of purchase at price 0)
- p = 10 (median observed price)
- y = 0.289707 (purchase probability at price 10)

The parameters are replaced in (12) as follows:

$$0.29 = \frac{1.9}{e^{10ß} + 1} \tag{12}$$

Then both sides of the equation (12) are rewritten in (13):

$$0.29 = \frac{29}{100} \text{ and } \frac{1.9}{e^{10ß}+1} = \frac{19}{10\left(e^{10ß}+1\right)}:$$
$$\frac{29}{100} = \frac{19}{10\left(e^{10ß} + 1\right)} \tag{13}$$

Next the equality is reversed to isolate b to the left-hand side of the equation (14).

$$\frac{19}{10\left(e^{10ß} + 1\right)} = \frac{29}{100} \tag{14}$$

Then both sides of the equation are multiplied by a constant (15).

Multiply both sides by $\frac{10}{19}$:

$$\frac{1}{e^{10ß}+1} = \frac{29}{190} \tag{15}$$

Then reciprocal of both sides of the equation is taken to solve for ß in the numerator (16).

$$e^{10ß} + 1 = \frac{190}{29} \tag{16}$$

Next, in (17) 1 is subtracted from both sides of the equation.

$$e^{10ß} = \frac{161}{29} \tag{17}$$

In (18) the natural logarithm is used on both sides to eliminate the exponential from the equation.

$$10ß = \log\left(\frac{161}{29}\right) \tag{18}$$

In (19) the β coefficient is obtained and it is equal to $\approx\ 0.171411$

$$ß = \frac{1}{10}\log\left(\frac{161}{29}\right) \tag{19}$$

Having α and ß, for each specified price point, we can determine the corresponding purchase probability, denoted as "p". Subsequently, we can input "p" into a Bernoulli random variable generator to simulate a customer's response to a specific price offer.

In Figure 14, the demand functions is generated for a set of points between 0 and 20 with parameters α = 1.9 and ß = 0.17.

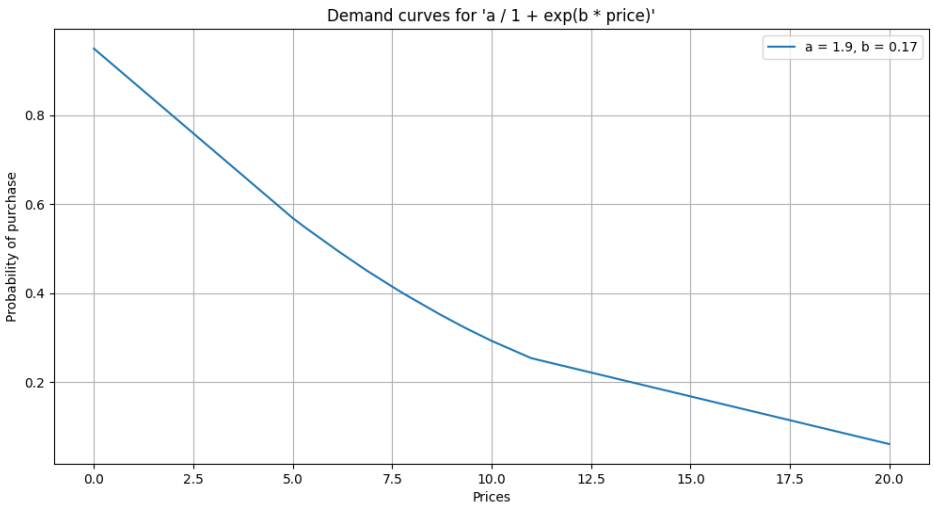

Figure 14. Demand curve, Source: Author (2024)

3.5.2 Reward function

To get the reward function the demand equation is multiplied by the price in the numerator (20).

$$y = \frac{p \,.\, \alpha}{1 + exp^{(ß \,.\, p)}} \tag{20}$$

The reward function does not achieve its maximum value corresponding to the highest probability, Figure 15. Additionally, the

price linked with the maximum value does not rely on the parameter α, whereas the maximum expected return does.

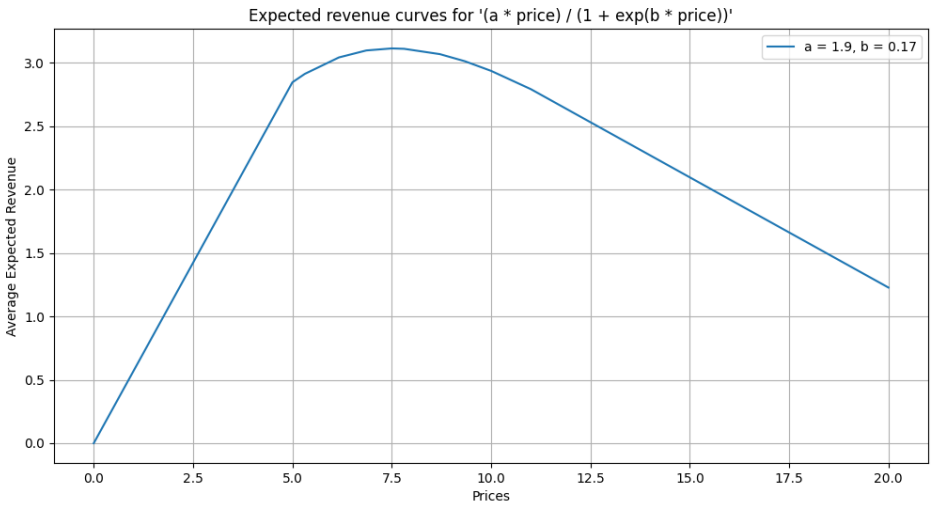

Figure 15. Expected revenue curve, Source: Author (2024)

3.5.3 Optimal price

Through the application of calculus techniques, specifically employing a combination of the product rule and the chain rule, we derive the formula for the derivative. This formula, when used alongside numerical algorithms, aids in determining the optimal price (21).

$$y = a \,.\, \left(\frac{1}{1 + exp^{(ß \,.\, p)}} - \frac{b \,.\, p \,.\, exp^{(ß \,.\, p)}}{(1 + exp^{(ß \,.\, p)})^2} \right) \tag{21}$$

This derivative function gives the exact price that maximizes our expected revenue curve. Employing this formula alongside numerical algorithms enables us to effectively identify the price that equates to zero. Consequently, this price corresponds to the one that maximizes the expected revenue.

3.6 Metrics for evaluation

The concept of employing the chosen metrics for evaluating the bandits is derived from Marmerola (2017), and Blum and Mansour (2007). They suggest the usage of three KPIs to identify which is the optimal algorithm: regret, reactivity, and arm allocation.

3.6.1 Regret

The concept of "regret" in the context of decision-making and algorithm analysis has been widely discussed and utilized in the fields of machine learning, game theory, and optimization by many practitioners. In the context of multi-armed bandit problems, the notion of regret is central to assessing how well an algorithm performs compared to an optimal strategy. Regret (22) measures the difference between the selected reward and the reward that could have been achieved by choosing the best possible action. The regret equation is:

$$Regret(t) = Optimal\ reward(t) - Actual\ reward \qquad (22)$$

Accumulating regret over time provides valuable insights into the extent of missed opportunities by not consistently selecting the best action. It is favored over cumulative reward as it provides a more transparent assessment of the algorithm's performance in relation to the optimal scenario.

3.6.2 Arms allocation

Arms allocation displays the distribution or assignment of resources, typically represented by arms or options, to optimize a particular objective. In dynamic pricing scenarios, arms represent different strategies or actions that the algorithm can choose. The arms allocation is presented in percentage, and it displays the algorithm's tendency to choose each arm throughout the duration. The max arm value is 100%, this is an example of optimal performance as the regret is going to be equal to 0.

3.6.3 Reactivity

This metric assesses the rate at which an algorithm converges towards a target average reward. In essence, it quantifies the adaptability and efficiency of the algorithm's learning process. A faster achievement of the desired average reward indicates greater responsiveness, suggesting a more rapid adjustment to the optimal price point.

The master thesis sets the target average reward at 95% of the optimal average reward. As initial steps may experience high variability the optimal average reward must be reached 10 times, excluding the first 100 episodes.

3.7 Price candidates

The pricing strategy involves a blend of historically used prices and newly introduced, unobserved prices. Initially, the thesis establishes the median historical price and computes the average

purchase probability associated with that price. Subsequently, for the chosen price point, the demand equation is solved to find the beta parameter. The price set is augmented by randomly generating candidates around the known prices. This results in a total of 10 prices, comprising 3 known prices and 7 newly created ones. Observed historical prices are [7.5, 10, 11], and around them, random selections are made. The lowest feasible price is set at the breakeven price of 5.00 euros, while the remainder falls within the range of 5 to 10 euros. Prices exceeding 11 euros are excluded, given their low probability and the lack of exploration demand for the current problem. The demand curve is solved using the prices, α and ß parameters to obtain the purchase probabilities for all prices, Table 1 gives overview on the selected discrete prices (arms) and associated probabilities for the multi armed bandit problem in stationary environment.

Price	Probabilities of purchase
5	0.5661
5.31	0.5452
6.17	0.4898
6.86	0.448
7.5	0.4115
7.81	0.3947
8.71	0.3486
9.34	0.3189
10	0.29
11	0.2503

Table 1. Discrete prices (arms) and probabilities for the multi armed bandit, Source: Author (2024)

4 Empirical research

The experimental design is derived from previous papers that have empirically investigated MAB algorithms (Kuleshov & Precup, 2014). The goal is to illustrate algorithm performance by comparing them in plots. The following master thesis will rely on the metrics specified in the methodology section to identify the top-performing algorithm in stationary and non-stationary environments.

4.1 Simulation loop

Assessing Multi-Armed Bandit (MAB) algorithms presents difficulties owing to the notably stochastic nature of their results. Because of the unpredictability in determining outcomes, the findings may significantly differ across iterations. To ensure a reliable evaluation, the optimal strategy entails running the simulation multiple times, gathering outcomes and metrics from each iteration, and subsequently computing the mean. The first stage is to create a feedback loop to emulate the decision-making procedure (Figure 16)

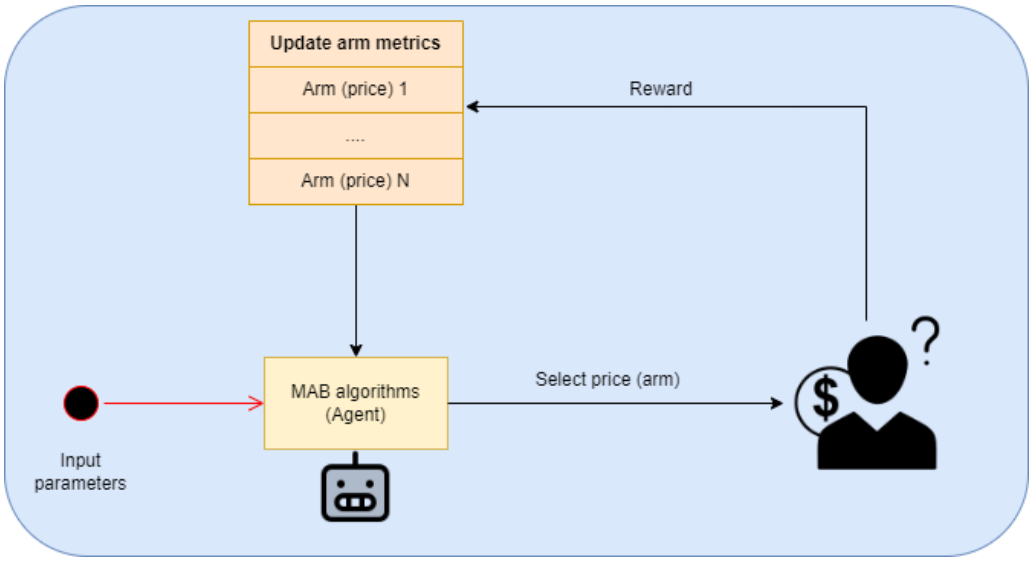

Figure 16. Decision making process – MAB problem, Source: Author (2024)

The input parameters for the process are:

- n_epochs – the total number of simulations (n_epochs = 100)
- n_steps (episodes) - the total number of episodes in a simulation. (n_steps = 15000)
- prices (arms) – discrete list of candidate prices [5.0, 5.31, 6.17, 6.86, 7.5, 7.81, 8.71, 9.34, 10.0, 11.0]

4.2 First phase: Algorithms performance in stationary environment

In this phase, the performance of the algorithms in section 3.4 is analyzed within a static environment. This environment is characterized by purchase probabilities that are defined once and remain unchanged throughout the evaluation. The optimal price for the static environment is determined to be 7.45, where the expected average return is highest.

Before delving into the results (Table 2), it is crucial to note that four experiments were conducted with the UCB algorithms. The first experiment, denoted as UCB1-1, employed a parameter C = 1 without any data normalization (see 3.4.4.1). The remaining three variations of the UCB1 algorithm involved normalization and utilized different values of C (0.5, 1, and 1.5). Another algorithm with varying hyperparameters is the epsilon-greedy algorithm, which was tested with two different values of epsilon (0.1 and 0.2).

Algorithm	Regret
ucb1-1	611.46 ± 797.83
ucb1-0.5-norm	662.62 ± 520.16
eps_greedy-0.1	707.49 ± 629.71
eps_greedy-0.2	832.47 ± 493.2
Thompson sampling	998.33 ± 512.5
ucb1-1-norm	1009.13 ± 347.07
ucb1-1.5-norm	1216.89 ± 516.84
greedy	1569.1 ± 1606.82

Table 2. Mean cumulative regret ± standard deviation after 100 epochs (15 000 episodes each) – stationary environment, Source: Author (2024)

Additionally, Figure 17 illustrates the timeline of the average cumulative regret over the episodes. As evident from Table 2 and Figure 17, the UCB-1-1, UCB-1-0.5-norm, and Epsilon-Greedy-0.1 are the top-performing algorithms in the current environment. Conversely, the greedy algorithm exhibits the least desirable performance. An interesting observation in the static environment is that normalization is unnecessary for the UCB-1 algorithm, and there's a preference for exploitation over exploration, as indicated by lower values of C.

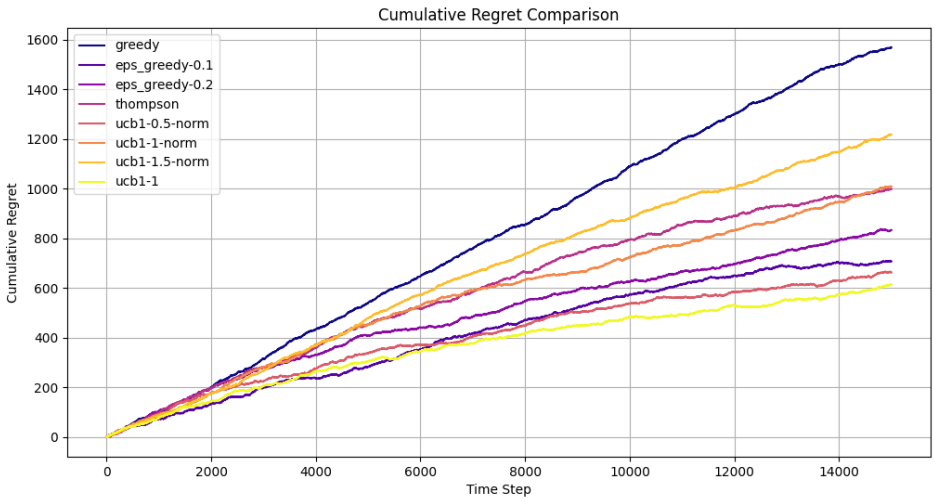

Figure. 17 Cumulative regret comparison, Source: Author (2024)

In terms of median regret, the UCB-1-1 algorithm emerges as the clear winner. However, when considering standard deviation, the UCB-1-0.5 algorithm demonstrates more precise estimates (see Figure 18).

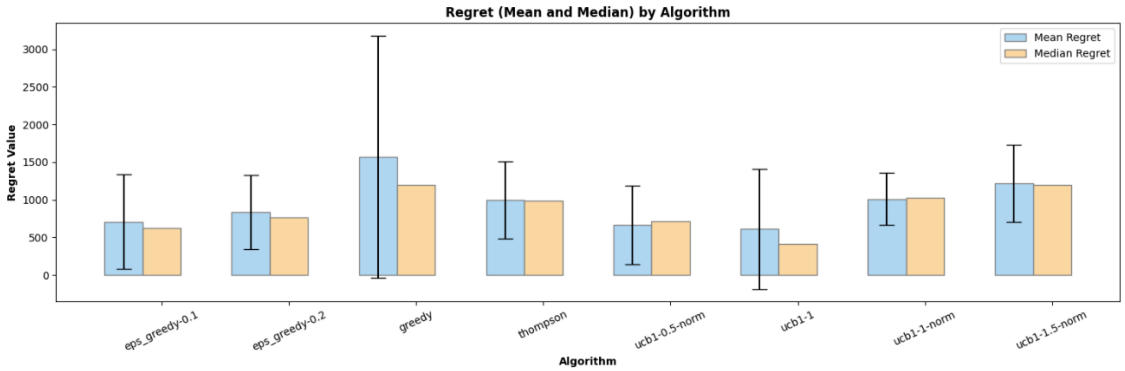

Figure 18. Regret (mean, median, and standard deviation), Source: Author (2024)

Conclusions cannot be drawn from the reactivity plot (Figure 19) as all algorithms reach it, on average, within 130 episodes.

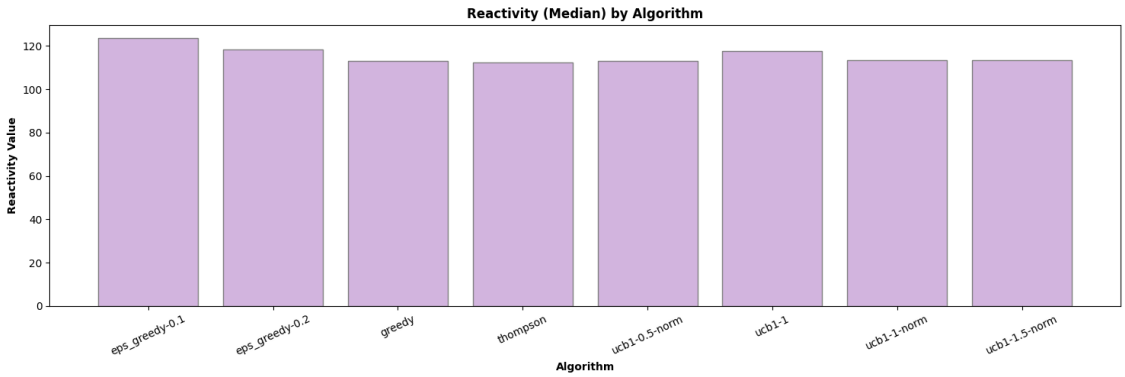

Figure 19. Reactivity stationary environment, Source: Author (2024)

The arm allocation, as shown in Figure 20, reveals that UCB1-0.5-norm and UCB1-1 algorithm successfully identify the best arm (price) in 25.3% and 23.6% of cases, respectively.

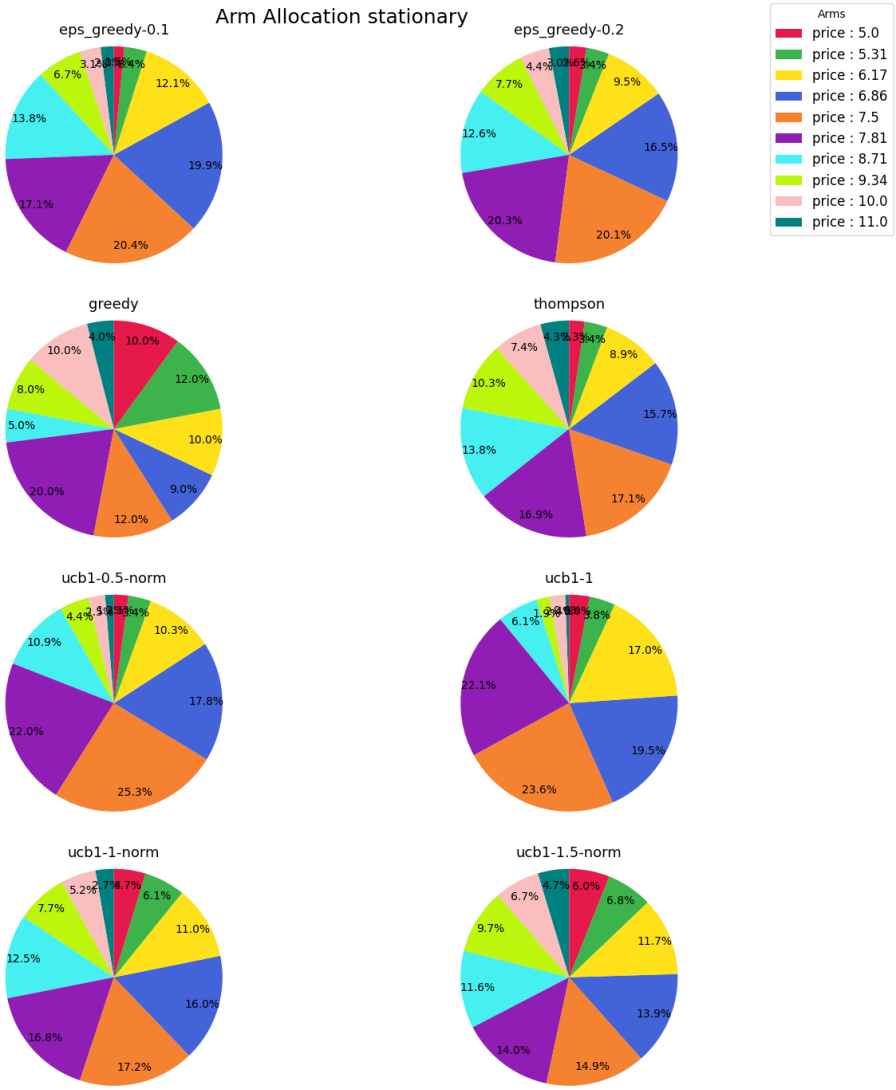

Figure 20. Arm allocation in stationary environment, Source: Author (2024)

4.3 Second phase: Non-stationary environment

In section 2.1.11, we briefly discussed the distinction and significance of stationary and non-stationary environments. In real-life scenarios, non-stationary environments are far more common, where factors such as seasonality, competition, marketing strategies, and other external influences can impact purchase probabilities. Therefore, this section introduces a partially changing probability

distribution to evaluate the performance of the algorithms tested in section 4.2.

In terms of the simulation environment, midway through each epoch, a competitor reduces the selling price of the same item to a fixed value of 6.5 euros.

As a result of this price reduction, the purchase probability decreases for prices equal to or higher than the competitor's price, gradually declining by a stepwise factor. The process begins by randomly determining the initial decreasing factor, with a value between 1.1 and 1.15. This initial decreasing factor is applied so that the price closest to or higher than the competitor's price of 6.5 euros gets reduced by this factor. Subsequently, each subsequent price is decreased by the previous factor multiplied by 20%. The concept is that the higher the price, the greater the decrease applied. The stepwise decrease factor is illustrated in Table 3.

Price	Denominator	Initial probability	Adjusted probability
5.00	1.00	0.5661	0.5661
5.31	1.00	0.5452	0.5452
6.17	1.13	0.4898	0.4350
6.86	1.35	0.4480	0.3316
7.50	1.62	0.4115	0.2538
7.81	1.95	0.3947	0.2029
8.71	2.33	0.3486	0.1493
9.34	2.80	0.3189	0.1138
10.00	3.36	0.2900	0.0863
11.00	4.03	0.2503	0.0620

Table 3, Adjustment of probabilities for the non-stationary environment, Source: Author (2024)

The aim of introducing the non-stationary environment is to assess the adaptability of the algorithms and evaluate how quickly they can adjust to changing conditions.

In terms of setup, it is important to highlight the distinctions between the first and second halves of the simulation in terms of the purchase probability. The first half utilized the values from the **'initial probability'** column in Table 3 for the set of prices, whereas the second half utilized those from the **'adjusted probability'** column. The optimal price for the first half is 7.45, whereas for the second half, it is 6.17.

Table 4 displays the mean cumulative regret after 100 epochs (15 000 episodes each) in a non-stationary environment.

Algorithm	Regret
ucb1-1	488.5 ± 1259.85
ucb1-0.5-norm	505.38 ± 561.53
eps_greedy-0.1	1064.95 ± 422.66
eps_greedy-0.2	1619.78 ± 408.63
Thompson sampling	1700.6 ± 907.67
ucb1-1-norm	2048.86 ± 747.54
ucb1-1.5-norm	2202.65 ± 679.7
greedy	6631.78 ± 6144.17

Table 4. Mean cumulative regret ± standard deviation after 100 epochs (15 000 episodes each) – non-stationary environment, Source: Author (2024)

Additionally, Figure 20 depicts the timeline of the average cumulative regret over the episodes. Notably, the vertical red line at step 7500 marks the change in the environment, where the optimal price shifts from 7.45 to 6.17 in response to the competitor's price change. Despite being designed for static environments, most of the algorithms demonstrated adaptability and eventually reached similar regret levels as observed in the static environment run by the end of the episodes.

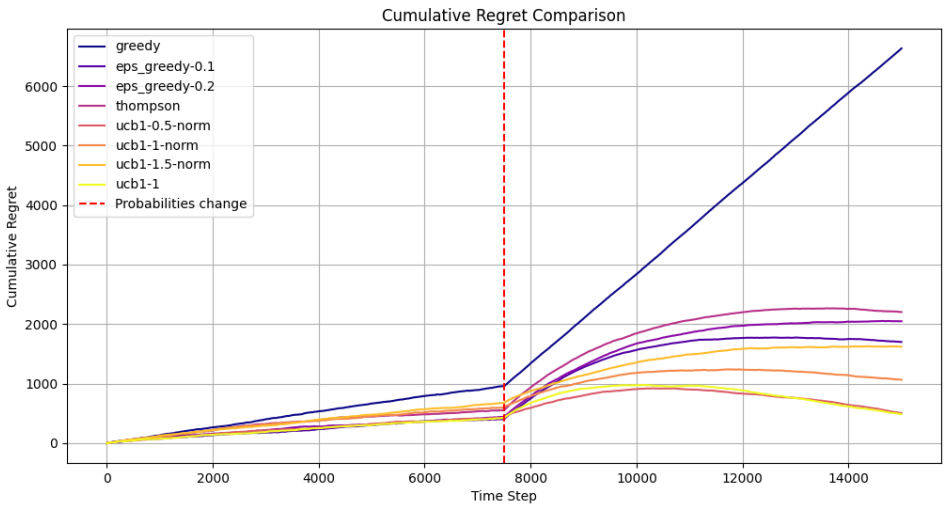

Figure. 20 Cumulative regret comparison, non-stationary environment, Source: Author (2024)

While the UCB1-1 algorithm leads the standings in terms of mean regret, it exhibits higher standard deviation compared to the ucb1-0.5-norm algorithm, which ranks second in the standings (see Figure 21).

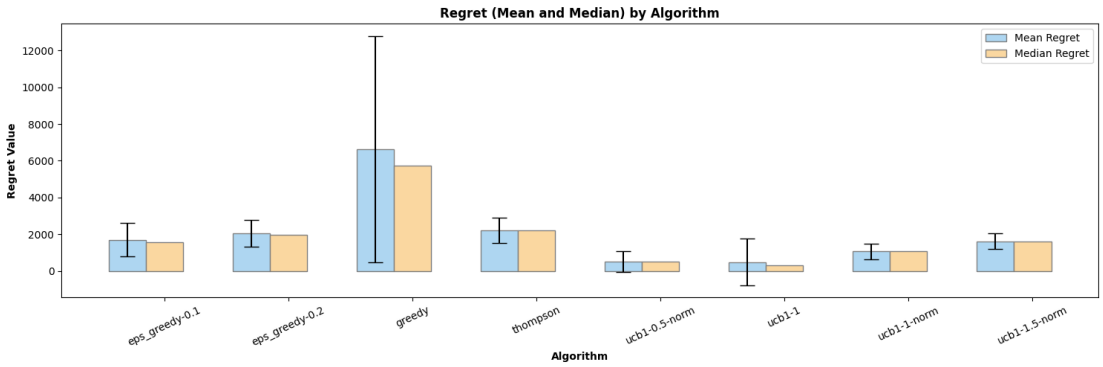

Figure 21. Regret (mean, median, and standard deviation), Source: Author (2024)

Notably, the performance of Thompson sampling is unsatisfactory. Expectations for the algorithm were higher, anticipating it to manage the problem more effectively and yield less

regret. However, it turned out to be ill-suited for the task, ranking quite low in performance.

The arm allocation in Figure 22 illustrates how the algorithms selected prices before the probability adjustment, while Figure 23 displays the allocation after the probability adjustment.

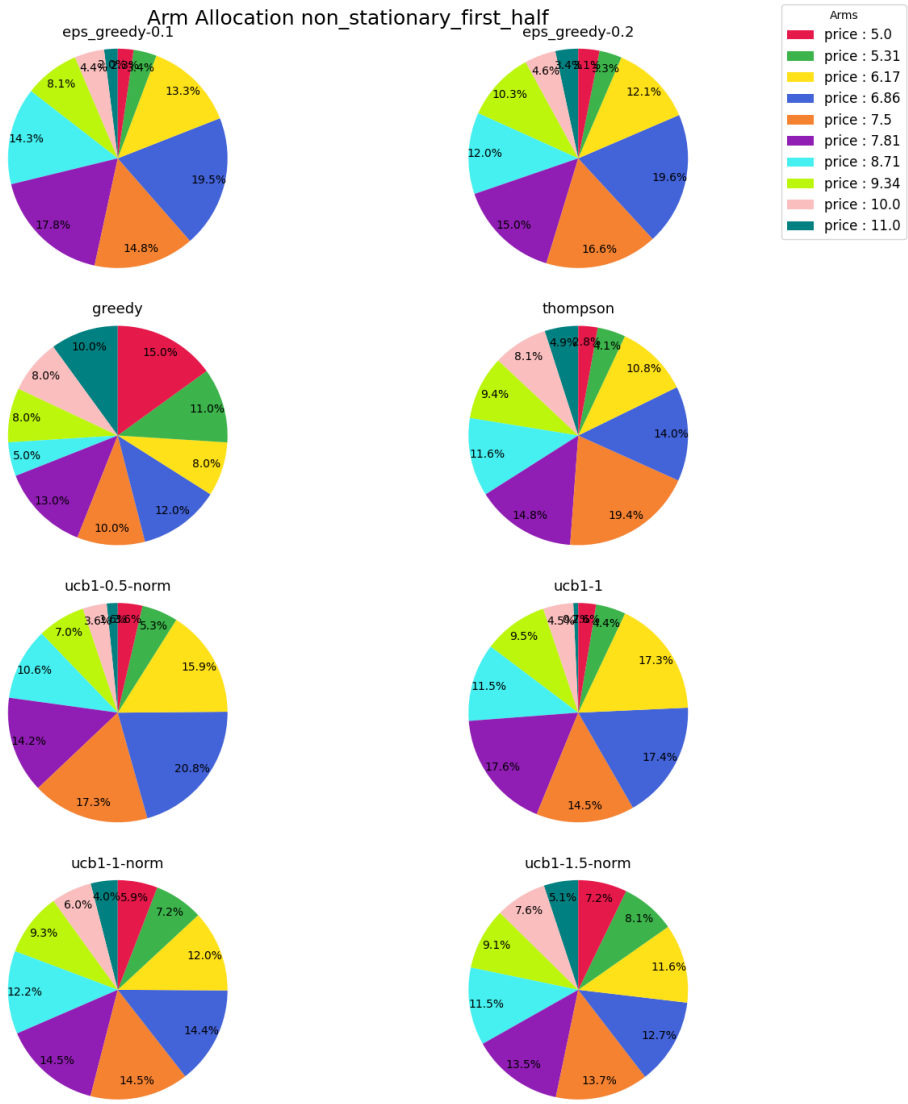

Figure 22, Arm allocation in the first 7500 episodes in a non-stationary environment, Source: Author (2024)

Figure 22 reveals that UCB1-0.5-norm and UCB1-1 algorithms successfully identify the arm that is closest to the optimal price (price

equal to 7.5 euro) in 17.3% and 14.5% of cases, respectively. Compared to the static environment, this indicates that with more episodes (15 000 in the static environment versus 7500 in the dynamic environment) under the same settings, the algorithms can learn better the optimal arm. Moreover, it turns out that 7500 episodes are not enough for the algorithms to establish a solid knowledge of the environment as they are selecting the arm with a price equal to 7.81 euros with 20.8% and 17.4% respectively. Although this arm is close it does not yield the optimal reward.

The arm allocation in the second half of the experiment, Figure 23, displays that the UCB1-0.5-norm and UCB1-1 are again quite reasonable. They managed to get fast to a satisfactory solution by selecting the arm with a price equal to 5.31 euros, although sub-optimal as the best price is at the arm that equals to 6.17 euros. Notably, these algorithms significantly outperform the baseline algorithm, "greedy."

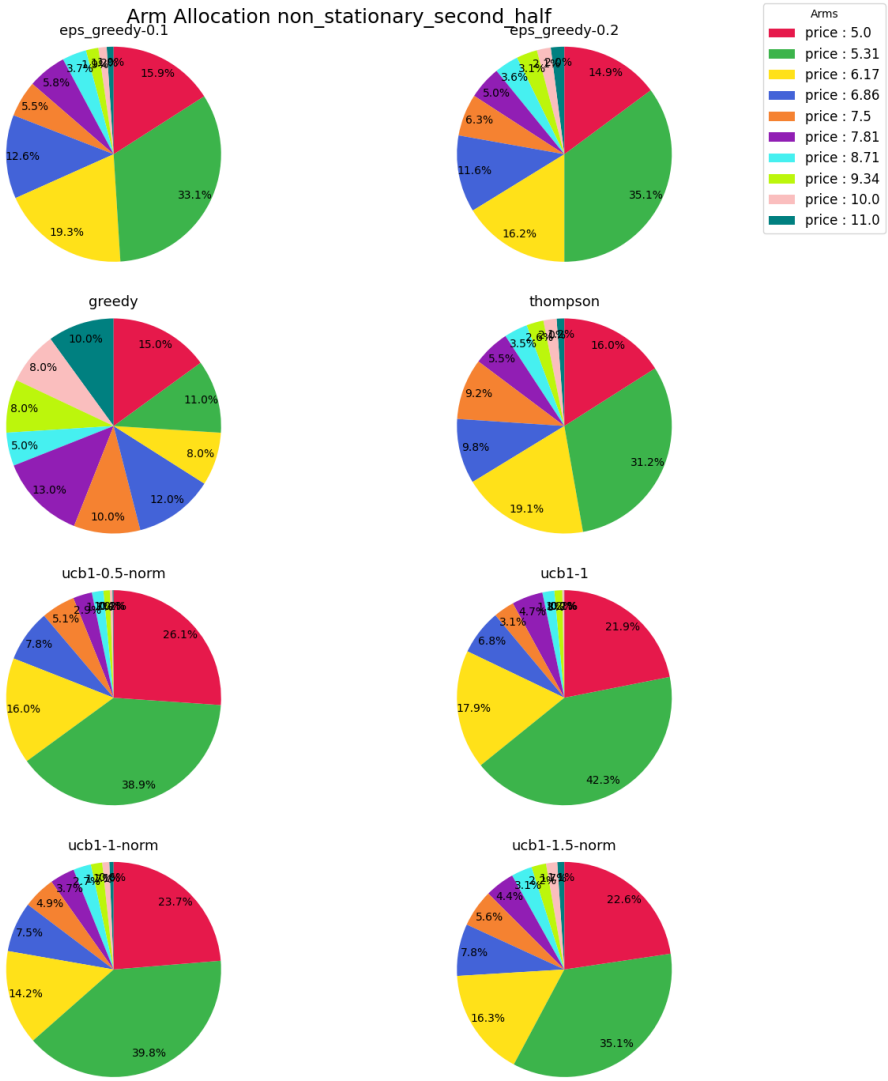

Figure 23, Arm allocation in the last 7500 episodes in a non-stationary environment, Source: Author (2024)

5 Conclusions

The thesis successfully formulates the dynamic pricing problem as a Bernoulli Multi-Armed Bandit Problem leveraging the principles of exploration and exploitation to design effective pricing strategies. Central to this endeavor were several pivotal tasks, each meticulously designed to contribute towards achieving the overarching objective of enhancing revenue optimization strategies for one of Europe's largest online food retailers.

Firstly, the thesis focused on the implementation and adaptation of four prominent Bernoulli multi-armed bandit algorithms: greedy, epsilon-greedy, Thompson sampling, and UCB1. Recognizing the unique intricacies of the online food retailing domain, these algorithms underwent careful adjustments to ensure their compatibility and effectiveness within this specific context. This task required a deep understanding of both the theoretical foundations of these algorithms and the practical nuances inherent in the online food retailing industry.

Secondly, a resilient global demand function and reward mechanism were developed to provide the algorithms with the necessary guidance for optimal decision-making. Given the dynamic nature of consumer behavior and market conditions in online food retailing, crafting a robust framework for estimating demand and assigning rewards was paramount. This task involves intricate data analysis techniques and domain expertise to accurately model customer preferences and purchase probabilities. The thesis utilizes a logit function to simulate purchase probabilities for a given set of prices, necessitating the determination of alpha and beta parameters. Given that the steepness (beta) of the curve may differ across products, it becomes imperative to ascertain the optimal value for the

parameter. This optimization process involves solving the demand function for beta, while holding the alpha parameter constant, based on historical median prices and corresponding purchase probabilities. By fine-tuning the beta parameter in this manner, the model can accurately reflect the nuanced dynamics of consumer behavior and market preferences, thereby enhancing the reliability and effectiveness of the simulated purchase probabilities. With the demand function at our disposal, we can readily derive the optimal price and probability by computing the derivative of the function. This approach allows to establish the optimal average expected return, which serves as a guiding principle for identifying the best arm during simulation.

The subsequent task involved the design and execution of experiments tailored to evaluate the performance of the implemented algorithms. Careful consideration was given to defining the parameters and conditions under which these experiments would be conducted. The experiments spanned various dimensions, including the range of price points, purchase probabilities, and the duration of pricing experiments. The fine-tuning of the algorithms' input parameters was undertaken to strike a delicate balance between exploration and exploitation. All these endeavors laid a solid foundation for conducting a comprehensive assessment of algorithmic performance.

Following the execution of experiments, a rigorous evaluation of each algorithm was undertaken based on predefined metrics: regret, reactivity, and arm allocation. This critical task involved analysis of experimental results to obtain meaningful insights into the strengths and limitations of each algorithm. The experimental results, spanning stationary and non-stationary environments, have

conclusively highlighted the robust performance and adaptability of the UCB1-0.5-norm algorithm, which emerged as the top performer among the tested algorithms. The algorithm showed adaptability in simple environments within the context of a single stock-keeping unit. Notably, the UCB1-0.5-norm algorithm consistently demonstrated low mean and median regret values, coupled with a commendably low standard deviation to the mean.

In essence, the thesis undertook a multifaceted approach to address the complex challenges inherent in dynamic pricing optimization within the online food retailing domain. Through a combination of theoretical insights, algorithmic implementations, empirical evaluations, and practical recommendations, the thesis sought to advance the understanding of how Bernoulli multi-armed bandit algorithms can be effectively leveraged to drive revenue optimization in one of the most competitive sectors of e-commerce.

References

Alabi, T. H. (2023, May 12). What is a Greedy Algorithm? Examples of Greedy Algorithms. freeCodeCamp.org. https://www.freecodecamp.org/news/greedy-algorithms/

An introduction to Reinforcement Learning. (n.d.). Reinforcement-Learning. https://deepanshut041.github.io/Reinforcement-Learning/notes/00_Introduction_to_rl/

Auer, P., Cesa-Bianchi, N., Freund, Y., & Schapire, R. E. (2000). Gambling in a rigged casino: The adversarial multi-armed bandit problem. *Electronic Colloquium on Computational Complexity*, *7*. http://eccc.hpi-web.de/eccc-reports/2000/TR00-068/index.html

Auer, Peter & Cesa-Bianchi, Nicolò & Fischer, Paul. (2002). Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning. 47. 235-256. https://link.springer.com/article/10.1023/A:1013689704352

Barrachina-Muñoz, S., Chiumento, A., & Bellalta, B. (2021). Stateless Reinforcement Learning for Multi-Agent Systems: the Case of Spectrum Allocation in Dynamic Channel Bonding WLANs. *2021 Wireless Days (WD)*, 1-5.

Blum, A., & Mansour, Y. (2007). Learning, regret minimization, and equilibria. In *Cambridge University Press eBooks* (pp. 79–102). https://doi.org/10.1017/cbo9780511800481.006

Bodea, T., & Ferguson, M. (2014). Segmentation, revenue management and pricing analytics. In *Routledge eBooks*. https://doi.org/10.4324/9780203802151

Carilli, T. (2020, August 31). *Chapter 8 Specifying Models | R companion to real Econometrics*. https://bookdown.org/carillitony/bailey/chp7.html

Chapelle, O., & Li, L. (2011). An empirical evaluation of Thompson sampling. *Neural Information Processing Systems*, *24*, 2249–2257. https://proceedings.neurips.cc/paper/2011/file/e53a0a2978c28872a4505bdb51db06dc-Paper.pdf

Greedy Algorithm. (n.d.). https://xlinux.nist.gov/dads//HTML/greedyalgo.html

Kocsis, L., Szepesvári, C. (2006). Bandit Based Monte-Carlo Planning. In: Fürnkranz, J., Scheffer, T., Spiliopoulou, M. (eds) Machine Learning: ECML 2006. ECML 2006. Lecture Notes in Computer Science(), vol 4212. Springer, Berlin, Heidelberg. https://doi.org/10.1007/11871842_29

Kuleshov, V., & Precup, D. (2014). Algorithms for multi-armed bandit problems. Journal of Machine Learning Research, 1, 1-32. https://doi.org/10.48550/arXiv.1402.6028

Krugman, P., & Wells, R. (2012). *Economics*. Worth Publishers.

Lattimore, T., & Szepesvári, C. (2020). *Bandit Algorithms*. Cambridge: Cambridge University Press.

Liu, Y., Van Roy, B., & Xu, K. (2023). A definition of Non-Stationary bandits. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2302.12202

Marmerola, G. D. (2017, November 21). Introduction to Thompson Sampling: the Bernoulli bandit. *Guilherme's Blog*. https://gdmarmerola.github.io//ts-for-bernoulli-bandit/

Mankiw, N. G. (2016). *Principles of economics*. Cengage Learning.

Phillips, R. L. (2005). *Pricing and revenue optimization*. Stanford Business Books, an imprint of Stanford University Press.

Robbins, H. (1952). Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*,

58(5), 527–535. https://doi.org/10.1090/s0002-9904-1952-09620-8

Russo, D., Van Roy, B., Kazerouni, A., Osband, I., & Wen, Z. (2018). A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning*, *11*(1), 1–96. https://doi.org/10.1561/2200000070

Slivkins, A. (2019). Introduction to Multi-Armed Bandits. *Foundations and Trends in Machine Learning*, *12*(1–2), 1–286. https://doi.org/10.1561/2200000068

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An Introduction*. Second edition MIT Press.

Taywade, K., Goldsmith, J., Harrison, B., & Bagh, A. (2023). Multi-armed Bandit Algorithms for Cournot Games. *Research Square*. https://doi.org/10.21203/rs.3.rs-2928787/v1

Thompson, W. 1933. "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples". Biometrika. 25(3/4): 285–294.

Whittle, P. (1988). Restless bandits: activity allocation in a changing world. *Journal of Applied Probability*, *25*(A), 287–298. https://doi.org/10.2307/3214163

Yang, Z., Liu, X., & Lü, Y. (2022). Exploration, Exploitation, and Engagement in Multi-Armed Bandits with Abandonment. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2205.13566